

InviteQR motion & haptics spec

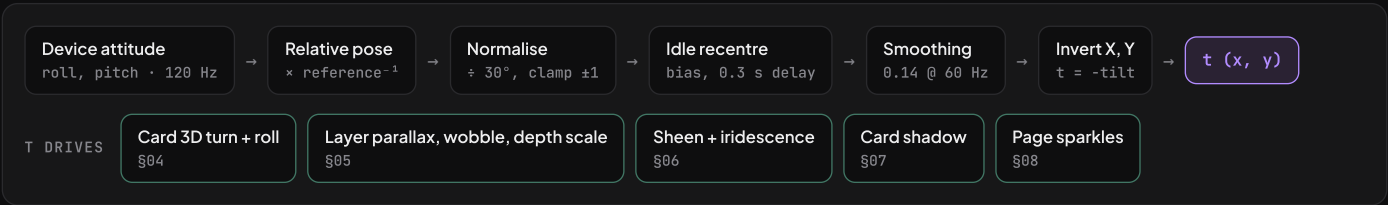
Everything that moves on the Invite share screen, how each value is derived from the phone's tilt, and every haptic on the screen. Numbers are the shipped defaults of the iOS prototype (SwiftUI), with React Native notes for each part.

Figma 980:15528 · 1 Child details Artboard 375 × 812 pt

Source GyroQR/ShareScreen.swift · GyroCardView.swift · MotionEngine.swift · CardSpec.swift · Tuning.swift · SlabDome.swift

01 Pipeline

One normalised tilt vector drives everything. The sensor pipeline produces `tilt = (x, y)`, both axes in `-1...1`. Both axes are then inverted, and every visual reads `t = (-tilt.x, -tilt.y)`, so the card leans *against* the phone's tilt. It is computed once per frame and read by every layer. Nothing on this screen animates on a timer: with the phone still, the card settles flat and stays there.



Screen layout: the 375 × 812 artboard is scaled to *cover* the device, `scale = max(W/375, H/812)`, and centred. All numbers below are artboard points.

02 Gyro input GYRO

CONSTANT	VALUE	MEANING
Sensor rate	120 Hz (8.3 ms)	Device-motion attitude updates. The per-frame step runs on the display link.
Reference frame	xArbitraryZVertical	iOS CoreMotion frame. The first attitude after start is stored as the reference, and every reading is taken relative to it(<code>attitude × reference⁻¹</code>).
<code>tilt.x</code> source	roll	Rotation about the device's long axis: tipping the left or right edge toward you.
<code>tilt.y</code> source	pitch	Rotation about the device's short axis: tipping the top edge toward or away from you.
Range	30°	Physical tilt that maps to a full ±1. <code>x = clamp(roll° / 30)</code> , same for y.
Smoothing	0.14 @ 60 Hz	Exponential low-pass, frame-rate independent (formula below).
Auto-recentre	on	Hold the phone still and the card eases back to flat, treating the new pose as level.
Still threshold	0.35 units/s	Speed of the raw sample (in tilt units) below which the phone counts as still.
Idle delay	0.3 s	Stillness needed before recentring starts.
Recentre half-life	0.45 s	Time for the bias to close half the remaining gap.
Invert X/Y	on / on	Every visual reads <code>t = (-tilt.x, -tilt.y)</code> , so the card leans <i>against</i> the device. The card rotation, parallax, light, shadow and sparkles all use the inverted value.

Per frame, in order

```
// dt = seconds since last frame, capped at 0.1
rel    = attitude relative to reference
sample = { x: clamp(rel.roll° / 30),
           y: clamp(rel.pitch° / 30) }

speed  = |sample - lastSample| / dt
idle   = speed < 0.35 ? idle + dt : 0
if (idle ≥ 0.3) {
  k     = 1 - 0.5^(dt / 0.45)
  bias += (sample - bias) · k
}
raw     = clamp(sample - bias)

a       = 1 - (1 - 0.14)^(dt · 60)
tilt    += (raw - tilt) · a

t       = { x: -tilt.x, y: -tilt.y } // invert X and Y
// every formula from §04 on reads t
```

Fallbacks

- **No gyro** (simulator, some Android devices): a demo sweep, `phase += dt·0.55`, `raw = (sin  $\varphi$ , 0.8·cos 0.73 $\varphi$ )`. The two frequencies are incommensurate, so the sweep never visibly repeats.
- **Touch drag** on the card: `raw = (dx / 140, -dy / 140)`, clamped ± 1 . On release `raw = 0` and the smoothing carries the card back to flat.
- **Reference pose**: captured the first time the sensor starts, and kept across later starts. The prototype also has a manual Recenter in its test panel, which takes the current attitude as the new reference and resets the bias to 0. For RN, capture the reference each time the screen gains focus, so the card always opens flat however the phone is held.

Start and stop: the sensor and the frame loop run only while this screen is visible. Start them on focus and stop them on blur.

03 Layer stack

How the screen and the card are put together, back to front. Each row is one layer. **Moves** says what the tilt does to it. **Blend**, **Clip** and **Mask** say how it combines with what is under it. Build it in exactly this order: the light only looks right because it lands on the finished card underneath it.

Screen (375 × 812 artboard)

Z	LAYER	ASSET / CONTENT	FRAME (X, Y, W, H)	MOVES	BLEND
1	Background	<code>inv_bg</code>	0, 0, 375, 812	<code>STATIC</code>	normal
2	Title and subtitle	"Invite Kiaan", H40 ExtraBold with gradient and white 4.8 pt outline · B14 Medium subtitle	57, 112.31, 261, 48 · 32, 162.31, 311.28, 20	<code>STATIC</code>	normal
3	Left sparkle	<code>inv_star_l</code>	-19.85, 459.20, 85, 89	<code>GYRO</code> drift, \$08	normal
4	Card (the stack below, as one unit)	—	42.17, 196.57, 290, 409.19	<code>GYRO</code> 3D turn and roll, \$04	normal, with drop shadow
5	Right sparkle	<code>inv_star_r</code>	250.67, 256.79, 153, 182.33	<code>GYRO</code> drift, \$08	normal
6	Close button	40 pt circle, white 50% fill, white 64% 1 pt ring, <code>ic_close</code> 20 pt	12, 55, 40, 40	<code>STATIC</code>	normal
7	Bottom block	OR row, link field (<code>ic_copy</code>), Share invite button (<code>ic_share</code>) and home bar. Only the button row and home bar are filled white.	0, 590, 375, 222	<code>STATIC</code>	normal

Card, inside z 4

Z	LAYER	ASSET / CONTENT	MOVES	BLEND	CLIP & MASK
4.1	Frame (chrome card)	<code>inv_frame</code>	turns with card only	normal	Everything from 4.1 to 4.6 is one group, clipped to the card box (290 × 409.19, no corner radius), then masked by the alpha of <code>inv_card_mask</code> , the chrome's own silhouette. Nothing inside can paint outside the chrome's outline.
4.2	Panel (recessed)	<code>inv_panel</code>	<code>GYRO</code> parallax h -3, depth scale	normal	
4.3	QR and nano badge	<code>inv_qr</code> with contact shadow	<code>GYRO</code> parallax h +7, wobble, depth scale, shadow offset	normal	
4.4	Name	live text "Kiaan Khalid"	<code>GYRO</code> parallax h +3, wobble	normal	
4.5	Specular sheen	white gradient band, 1181.5 pt square	<code>GYRO</code> slides -t × 260, opacity with t	additive (plusLighter)	
4.6	Iridescence	rainbow gradient band, 1181.5 pt square	<code>GYRO</code> slides +t × 300, opacity with t	additive (plusLighter)	

- **Sheen and iridescence sit on top of every card layer**, the QR and the name included, with the iridescence above the sheen. Both add light to the finished card beneath them. Blending them against anything else, such as one layer or an empty canvas, gives the wrong colour.
- **Clip before mask.** The bands are far larger than the card. Clip them to the card box first, then apply `inv_card_mask`, so the chrome's rounded corners and outer glow shape the light. Without the mask, a pale rectangle shows where the box extends past the rounded chrome.

- **Composite, then shadow, then transform.** Flatten 4.1–4.6 into one image, cast the card's drop shadow (§07) from that image, and only then apply the roll, pitch and yaw (§04). If the layers are turned separately, their clips can drift apart under perspective and light can leak past the card's edge.
- **No rim light or halo** on this card. The chrome artwork carries its own edge and glow.

```
// React Native Skia: card group, back to front
<Canvas style={{ width: 290, height: 409.19 }}>           // wrapper gets the 3D transform + shadow
  <Group clip={rect(0, 0, 290, 409.19)}>
    layer=<Paint></Paint>>                                // offscreen group, flattened before the transform
    <Mask mode="alpha" mask=<Image image={cardMask} fit="fill" width={290} height={409.19} />>
      <Image image={frame} x={0} y={0} width={290} height={409.19} />
      <Image image={panel} x={39.33 + px(-3)} y={42.19 + py(-3)} width={212} height={244} />
      <Image image={qr}    x={61.50 + px(7)}  y={77.43 + py(7)}  width={172} height={173} />  // + wobble, shadow
      <Text  ... name at 20.53 + px(3), 305.13 + py(3) />
      <Rect  ... sheen band  blendMode="plus" opacity={sheenOpacity} />
      <Rect  ... holo band   blendMode="plus" opacity={holoOpacity} />
    </Mask>
  </Group>
</Canvas>
```

04

Card transform

GYRO

The card is 290 × 409.19 pt, placed at (42.17, 196.57) on the artboard. The whole card, with its layers and light already composited, is transformed as one unit. Transforms are listed innermost first, which is the order they apply to the card.

#	TRANSFORM	FORMULA	AT FULL TILT (1, 1)
1	Composite and drop shadow	see §07	—
2	In-plane roll (rotateZ)	<code>tilt.x</code> × 2.2°	2.2°
3	Pitch (rotateX)	<code>tilt.y</code> × 19°	19°
4	Yaw (rotateY)	<code>tilt.x</code> × 19°	19°
5	Perspective	SwiftUI 0.85 ≈ RN perspective: 481	—
6	Scale	1.0	1.0

VERIFY **Perspective** is derived, not measured. SwiftUI's `rotation3DEffect` sets `m34 = -p / max(w, h)`, so `p = 0.85` on a 409 pt card gives a camera distance of about 409 / 0.85 ≈ 481 pt. Put the RN build next to the iOS prototype at full tilt and adjust it by eye. **Signs:** rotateX and rotateY conventions differ between SwiftUI and RN. With both axes inverted, tipping the phone's right edge away brings the card's right edge *toward* the viewer, and tipping its top edge away brings the card's top edge toward the viewer. Calibrate the RN build to that, against the iOS prototype.

In every formula from here on, `tilt.x` and `tilt.y` mean the inverted value `t` from §02. `tiltMagnitude = min(1, √(x² + y²))` is used by the depth scale, the sheen, the iridescence and the shadow radius, and is the same either way.

05

Layers & parallax

GYRO

Depth model: a layer `h` pt above the card's surface is shifted by `h · tan θ` before the card rotates. That one relationship makes the QR read as floating over its card. The recessed panel has a negative `h`, so it drifts against the tilt.

offset.x = h · tan(tilt.x · 19°) · offset.y = -h · tan(tilt.y · 19°) · tan 19° = 0.3443

LAYER	ASSET	FRAME IN CARD (X, Y, W, H)	ELEVATION H	PARALLAX AT FULL TILT	WOBBLE	DEPTH SCALE	CONTACT
Frame (chrome card)	<code>inv_frame</code>	0, 0, 290, 409.19	0	0	—	1	—
Panel (recessed)	<code>inv_panel</code>	39.33, 42.19, 212, 244	-3 pt	±1.03 pt	—	1 - 0.005· t	—

LAYER	ASSET	FRAME IN CARD (X, Y, W, H)	ELEVATION H	PARALLAX AT FULL TILT	WOBBLE	DEPTH SCALE	CONTACT
QR and nano badge	<code>inv_qr</code>	61.50, 77.43, 172, 173	+7 pt	±2.41 pt	<code>tilt.x × 1.0°</code>	<code>1 + 0.0117 · t </code>	black 12 blur 6, y +2, of -0.75 × parallax
Name "Kiaan Khalid"	live text, H28 Bold, gradient #000 → #4C17A0 from 15.4%	20.53, 305.13, 246.6, 36	+3 pt	±1.03 pt	<code>tilt.x × 0.5°</code>	—	—

- **Wobble** is an extra in-plane rotation of that one layer: `tilt.x × 5° × layer.wobble` (0.2 for the QR, 0.1 for the name).
- **Depth scale**: nearer planes grow slightly as the card turns, `1 + (h / 600) · tiltMagnitude`. It applies to image layers only, not the name.
- **Contact shadow** (QR only): `x = -0.75 · offset.x`, `y = 2 - 0.75 · offset.y`. It stays opposite the float, so the QR looks lit from the front.
- **Clipping**: the frame, panel, QR, name and light bands are clipped to the card box, then masked by `inv_card_mask`'s alpha (the chrome's own silhouette).

06

Light

GYRO

Two gradient bands slide across the face, both drawn with an additive blend (iOS `plusLighter`, Skia `BlendMode.Plus`). Each is a square of side `diag(card) + 2 × 300 + 80 = 1181.5 pt`, larger than the card so its straight edges never slide into view. Both are clipped to the card and `inv_card_mask`.

Specular sheen		Iridescence (holo band)	
Direction	top-left → bottom-right (45°)	Direction	top-left → bottom-right (45°)
Stops (white α @ %)	0 @40.7 · .22 @48.1 · .45 @50 · .22 @51.9 · 0 @59.3	Stops (HSB, s .85, b 1)	clear →41.5 · h0 @45 · h.13 @47.9 · h.33 @50 · h.52 @52.1 · h.72 @55 · h.88 @57.8 · clear @61.3→
Offset	(-tilt.x × 260, -tilt.y × 260)	Offset	(+tilt.x × 300, +tilt.y × 300), the opposite way to the sheen
Opacity	0.38 × (0.25 + 0.75 · t)	Opacity	0.16 × t
At rest	visible at 0.095	At rest	hidden (0), so the resting card matches Figma

Rim light: the prototype also has a rim highlight that sweeps along a card's border. It follows the tilt, but it is drawn on the card's stroke, and this card's border width is 0 (the chrome frame carries its own edge). So on the Invite card it draws nothing. Don't implement it for this screen.

07

Card shadow

GYRO

PROPERTY	FORMULA	REST	FULL TILT (1, 1)
Colour	<code>black × 0.10</code>	0.10	0.10
Blur radius	<code>18 + 6 · t </code>	18	24
Offsetx	<code>-tilt.x × 20</code>	0	-20
Offsey	<code>6 - tilt.y × 20</code>	6	-14

The shadow is applied to the composited card before the rotation, so it turns with the card. SwiftUI radius is a Gaussian σ -style blur. For RN's iOS `shadowRadius` start at the same value and compare; Android elevation cannot express an offset, so there use Skia or a pre-blurred image.

08 Page sparkles GYRO

The two chrome sparkles are page decoration: they don't rotate with the card, but drift a few points against it, which puts the card between them in depth.

SPARKLE	ASSET	FRAME ON ARTBOARD	STACKING	DEPTH D	OFFSET	FULL TILT
Left	inv_star_l	-19.85, 459.20, 85, 89	behind card	-9	(d·x·0.5, -d·y·0.5)	±4.5 pt
Right	inv_star_r	250.67, 256.79, 153, 182.33	in front of card	+14	(d·x·0.5, -d·y·0.5)	±7 pt

Everything else on the page stays still: the background (`inv_bg`), title, subtitle, OR row, link field, Share button and close button.

09 Haptics HAPTIC

The screen has one haptic source: the **Share invite** button. Tilt is continuous and has no haptics.

TRIGGER	IOS GENERATOR	INTENSITY	NOTES
Share invite: press down	<code>UIImpactFeedbackGenerator(.heavy)</code>	1.0	Fires on touch-down (<code>onPressIn</code>), not on the tap. Call <code>prepare()</code> ahead of time so the first press has no lag.
Share invite: release	<code>UIImpactFeedbackGenerator(.light)</code>	0.4	Fires on every release (<code>onPressOut</code>), including a cancelled press. It is softer on purpose: equal weights read as a double tap.
Close (x)	—	—	No haptic, no press animation.
Copy link	—	—	Not wired in the prototype yet. Suggested: light impact at 0.6, with a "Link copied" toast.
Card tilt / drag	—	—	None.

Share button press visual

Scale while held	0.97
Scalespring	SwiftUI response 0.26 s, damping 0.68 ≈ stiffness 584, damping 32.9, mass 1
Facelight swap	raised → sunk, 75 ms ease-in-out
Label opacity held	0.9
Drop shadow	0.5 strength

React Native mapping

- `react-native-haptic-feedback` : `impactHeavy` on press-in, `impactLight` on press-out. It has no intensity control, so the release will be slightly stronger than 0.4.
- For exact intensities, use a small native module calling `UIImpactFeedbackGenerator.impactOccurred(intensity:)` .
- Android: `HapticFeedbackConstants.VIRTUAL_KEY` for press-in and `KEYBOARD_TAP` for press-out, or `VibrationEffect.createOneShot(10, amplitude)` where amplitude is supported.
- Respect the app-wide haptics setting: every call is gated on it.

10 Calculator

Values below are at t = (1.00, 1.00), full tilt on both axes. The interactive version of this sheet (the HTML file) lets you set any tilt.

OUTPUT	VALUE
tiltMagnitude	1.000
Card yaw (rotateY)	+19.00°
Card pitch (rotateX)	+19.00°
Card roll (rotateZ)	+2.20°
QR offset (h +7)	+2.41, -2.41 pt

OUTPUT	VALUE
Panel offset (h −3)	-1.03, +1.03 pt
Name offset (h +3)	+1.03, -1.03 pt
QR wobble	+1.00°
Name wobble	+0.50°
QR depth scale	1.0117
Panel depth scale	0.9950
QR contact shadow	-1.81, +3.81 pt
Sheen offset	-260.0, -260.0 pt
Sheen opacity	0.380
Holo offset	+300.0, +300.0 pt
Holo opacity	0.160
Card shadow radius	24.00 pt
Card shadow offset	-20.0, -14.0 pt
Left sparkle offset	-4.50, +4.50 pt
Right sparkle offset	+7.00, -7.00 pt

11 React Native notes

1. **Sensor on the UI thread:** `useAnimatedSensor(SensorType.ROTATION, { interval: 8 })` from Reanimated 3. Keep the whole pipeline in worklets so tilt never crosses the bridge.
2. **Relative pose:** store the first quaternion as the reference and multiply each reading by its inverse, then read roll and pitch. Subtracting Euler angles also works inside $\pm 30^\circ$, but drifts near the edges.
3. **Frame loop:** run the step in `useFrameCallback` with the real `dt`, so smoothing and recentring behave the same at 60 and 120 Hz.
4. **Transforms:** `[{ perspective: 481 }, { rotateY: yaw }, { rotateX: pitch }, { rotateZ: roll }]` on the card wrapper. RN applies them right to left, which matches the order in §04.
5. **Light bands and masks:** use React Native Skia. A `LinearGradient` in a large `Rect` with `blendMode="plus"`, clipped by an `Image` mask of `inv_card_mask`. Draw the card layers and the bands into one Skia canvas, then apply the 3D transform to that canvas's wrapper. Blending and clipping then happen in the card's plane, and nothing leaks outside the card at an angle.
6. **Springs:** SwiftUI's `response / dampingFraction` maps to Reanimated as `stiffness = (2 $\pi$  / response)2` and `damping = 2 ·  $\zeta$  · (2 $\pi$  / response)`, with `mass 1`.
7. **Assets** are exported at 3× the frames listed: `inv_bg`, `inv_frame`, `inv_panel`, `inv_qr`, `inv_card_mask`, `inv_star_l`, `inv_star_r`, `ic_close`, `ic_copy`, `ic_share`.

```
// Reanimated sketch: the tilt worklet
const rot = useAnimatedSensor(SensorType.ROTATION, { interval: 8 });
const tilt = useSharedValue({ x: 0, y: 0 });
const s = useSharedValue({ ref: null, bias: {x:0,y:0}, last: {x:0,y:0}, idle: 0 });

useFrameCallback(({ timeSincePreviousFrame }) => {
  'worklet';
  const dt = Math.min(0.1, (timeSincePreviousFrame ?? 16.7) / 1000);
  const { roll, pitch } = relativeTo(s.value, rot.sensor.value); // quaternion * ref-1
  const clamp = v => Math.max(-1, Math.min(1, v));
  const sx = clamp(roll * 57.2958 / 30), sy = clamp(pitch * 57.2958 / 30);
  const speed = Math.hypot(sx - s.value.last.x, sy - s.value.last.y) / dt;
  const idle = speed < 0.35 ? s.value.idle + dt : 0;
  let { bias } = s.value;
  if (idle ≥ 0.3) {
    const k = 1 - Math.pow(0.5, dt / 0.45);
    bias = { x: bias.x + (sx - bias.x) * k, y: bias.y + (sy - bias.y) * k };
  }
  s.value = { ...s.value, bias, last: { x: sx, y: sy }, idle };
  const a = 1 - Math.pow(1 - 0.14, dt * 60);
  const rx = clamp(sx - bias.x), ry = clamp(sy - bias.y);
  tilt.value = { x: tilt.value.x + (rx - tilt.value.x) * a,
    y: tilt.value.y + (ry - tilt.value.y) * a };
});
// Invert X and Y: every visual reads t, not tilt.
const t = useDerivedValue(() => ({ x: -tilt.value.x, y: -tilt.value.y }));
```

12 **Tuning ranges**

The prototype's test panel exposes these knobs. The defaults are the spec; the ranges show how far each can go before it breaks the look. Worth keeping as remote config in RN.

KNOB	DEFAULT	RANGE	AFFECTS
Range (tilt for ±l)	30°	8–80°	Input sensitivity
Smoothing	0.14	0.02–1	Lag against jitter; l = raw
Idle delay	0.3 s	0.3–5 s	When recentring starts
Invert X/Y	on / on	on or off	Card leans against (on) or with (off) the device
Tilt amount	19°	0–40°	Card yaw/pitch and parallax
Perspective	0.85	0–1.5	3D depth of the turn
Roll	2.2°	0–8°	In-plane rotation
Elevation ×	1.0	0–3	All layer and sparkle depths
Contact shadow	1.0	0–2	QR shadow strength
Wobble	5°	0–20°	Per-layer in-plane rotation
Sheen	0.38	0–1	Specular band strength
Holo	0.16	0–0.8	Iridescent band strength
Card shadow shift	20 pt	0–45 pt	Shadow travel with tilt
Button press scale	0.97	0.85–1.0	Share button while held

13 **QA checklist**

- Phone held still for 0.3 s: the card starts easing back to flat and is most of the way there about 1 s later, with no snap.
- Right edge of the phone tipped away (inverted axes): the card's right edge comes toward you, the QR drifts left about 2.4 pt, the panel drifts right about 1 pt, and the sheen slides right.
- At rest, the iridescence is invisible and the sheen is faint (about 0.095).
- At any angle, no light shows outside the chrome silhouette, including at the corners.
- Share button: the heavy tick lands on touch-down, before the finger lifts, and a light tick follows on release.

- Leaving the screen stops the sensor: check battery or CPU in the profiler.
- Reduced Motion on: consider freezing tilt at 0 (the prototype doesn't yet).

Generated from the GyroQR iOS prototype on 28 Sep 2026. Values are the code defaults at that date. The Share button press haptics and visual come from the app-wide button (DomeButtonStyle). v3: idle delay 0.3 s, both axes inverted, layer stack added.